TECH REFERENCE

PLE + adaTT
Technical Reference

Progressive Layered Extraction · Adaptive Task Transfer
CGC Gate · GroupTaskExpertBasket · Logit Transfer · 2-Phase Training

Project
AWS PLE for Financial
PLE-Cluster-adaTT Architecture

Version
v1.0
2026-04-01

> This document describes the core structure of the PLE-Cluster-adaTT architecture. It covers the progression from Shared-Bottom to PLE, heterogeneous expert Basket design, adaTT gradient-based task transfer, Task Groups structure, training strategies, and FP16/GradScaler implementation considerations.

Table of Contents

1. PLE Architecture . . . 4
1.1 Motivation for Multi-Task Learning . . . 4
1.2 Progression from Shared-Bottom to PLE . . . 4
Shared-Bottom (Caruana, 1997) . . . 4
MMoE (Ma et al., KDD 2018) . . . 4
PLE (Tang et al., RecSys 2020) . . . 4
1.3 CGC (Customized Gate Control) . . . 5
CGCLayer (weighted sum approach) . . . 5
CGCAttention (block scaling approach) . . . 5
Initial Bias Setting (domain_experts) . . . 6
Entropy Regularization . . . 6
Dimension Normalization (v3.3) . . . 6
2. Heterogeneous Expert Basket . . . 7
2.1 Pool/Basket Pattern . . . 7
2.2 Expert Selection Criteria . . . 7
2.3 Expert Routing . . . 8
3. adaTT — Adaptive Task-aware Transfer . . . 9
3.1 Motivation: Negative Transfer . . . 9
3.2 Gradient Cosine Similarity . . . 9
3.3 EMA Stabilization . . . 9
3.4 Transfer-Enhanced Loss . . . 10
3.5 Transfer Weight Computation (4-stage pipeline) . . . 10
3.6 3-Phase Schedule . . . 11
3.7 Group Prior . . . 11
3.8 Negative Transfer Detection and Blocking . . . 11
3.9 Analogy to Attention Mechanism . . . 12
4. Task Groups . . . 13
4.1 Four Financial DNA Groups . . . 13
4.2 Intra/Inter Strength Design . . . 13
4.3 Three Logit Transfer Approaches . . . 13
4.4 GroupTaskExpertBasket (v3.2) . . . 14
Soft Routing . . . 14
5. Training Strategy . . . 15
5.1 2-Phase Training . . . 15
Phase 1: Shared Expert Pretrain . . . 15
Phase 2: Cluster Finetune . . . 15
5.2 Uncertainty Weighting . . . 15
5.3 Focal Loss . . . 16
5.4 AMP (Automatic Mixed Precision) . . . 16
6. Implementation Notes . . . 17
6.1 FP16 Considerations . . . 17
Focal Loss float32 Casting . . . 17

adaTT Gradient Extraction and autocast . . . 17
6.2 GradScaler Guard . . . 17
Gradient Extraction Frequency Optimization . . . 17
torch.compiler.disable . . . 17
6.3 Phase 2 Frozen Layers . . . 18
CGC-adaTT Co-freezing . . . 18
6.4 Full Loss Computation Pipeline Order . . . 18
6.5 Debugging Guide . . . 18
References . . . 20

Design vs. Implementation Note

This document is written based on the full-bank design (734D). The current Santander benchmark implementation is 349D raw input (13 feature groups), expanding to 403D after Phase 0 log1p expansion.

adaTT Scale Note (2026-04-15). adaTT degrades at 13-task scale. Loss-level transfer (156 task pairs) undoes PLE's representation-level separation, causing gradient interference. The active implementation replaces adaTT with **GradSurgery (PCGrad task-type projection)**, which respects PLE's expert boundaries. This document is retained as a reference for the adaTT design; see `grad_surgery.py` for the current implementation.

1. PLE Architecture

1.1 Motivation for Multi-Task Learning

The AIOps recommendation system must simultaneously predict 13 tasks. Leveraging shared patterns across tasks dramatically improves data efficiency. The total loss is defined as a weighted sum:

$$\mathcal{L}_{\text{MTL}} = \sum_{k=1}^{K} w_k \cdot \mathcal{L}_k\Big(f_k\Big(\boldsymbol{h}_{\text{shared}(\boldsymbol{x})}\Big)\Big)$$

$\boldsymbol{h}_{\text{shared}}$ is difficult to overfit to any single task. Only representations that are simultaneously useful to all tasks survive, which constitutes **inter-task regularization**.

1.2 Progression from Shared-Bottom to PLE

Shared-Bottom (Caruana, 1997)

All tasks share a single trunk and then branch into task-specific heads.

$$\boldsymbol{h} = f_{\text{shared}(\boldsymbol{x})} \quad \rightarrow \quad \hat{y}_k = f_k^{\text{tower}(\boldsymbol{h})}$$

Implementation is simple and parameter-efficient, but **Negative Transfer** is severe when task correlation is low.

MMoE (Ma et al., KDD 2018)

N experts with identical structure are maintained, and task-specific gates determine their weighted combination.

$$\boldsymbol{h}_k = \sum_{i=1}^{N} g_{k,i} \cdot f_i^{\text{expert}(\boldsymbol{x})}, \quad \boldsymbol{g}_k = \text{Softmax}\Big(\boldsymbol{W}_k^{\text{gate}} \cdot \boldsymbol{x}\Big)$$

Different expert combinations per task are possible, but the **Expert Collapse** problem arises where all gates select the same experts.

PLE (Tang et al., RecSys 2020)

Experts are explicitly separated into **Shared Experts** $\mathcal{E}^s$ and **Task-specific Experts** $\mathcal{E}^k$, and the CGC gate combines both types of experts at an optimal ratio.

$$\boldsymbol{h}_k = \sum_{i=1}^{|\mathcal{E}^s|} g_{k,i}^s \cdot \boldsymbol{e}_i^s + \sum_{j=1}^{|\mathcal{E}^k|} g_{k,j}^k \cdot \boldsymbol{e}_j^k$$

$\boldsymbol{e}_i^s$: output of i-th Shared Expert, $\boldsymbol{e}_j^k$: output of j-th Task Expert for task k
$g_{k,i}^s, g_{k,j}^k$: CGC gate weights (Softmax-normalized)

Problems PLE solves:

1. **Negative Transfer mitigation**: Task-specific Experts learn specialized patterns without interference
2. **Expert Collapse prevention**: Shared/Task Expert roles are naturally separated
3. **Progressive Extraction**: Stacking multiple layers refines information from low-level to high-level

Category	Shared-Bottom	MMoE	PLE
Expert Structure	Single shared trunk	N experts fully shared	Shared + Task-specific separated
Gating	None	Task-wise Softmax gate	CGC: Shared + Task Expert combined
Negative Transfer	High	Medium (Expert Collapse)	Low (explicit separation)
Expert Collapse	N/A	High	Low

1.3 CGC (Customized Gate Control)

This implementation has two CGC variants with different roles and output shapes.

CGCLayer (weighted sum approach)

The CGC from the original PLE paper, where task-wise gates produce a **weighted sum** of Shared + Task Expert outputs. Output dimension is fixed at `expert_hidden_dim` (independent of the number of experts).

$$\boldsymbol{h}_k = \sum_{i=1}^{N} g_{k,i} \cdot \boldsymbol{e}_i, \quad \boldsymbol{g}_k = \mathrm{Softmax}\left(\boldsymbol{W}_k^{\text{gate}} \cdot \boldsymbol{x}\right) \in \mathbb{R}^N$$

N: total number of Shared + Task Experts
$\boldsymbol{e}_i \in \mathbb{R}^{d_{\text{expert}}}$: output of i-th Expert
Result: $\boldsymbol{h}_k \in \mathbb{R}^{d_{\text{expert}}}$ — fixed dimension independent of the number of experts

CGCAttention (block scaling approach)

Outputs of heterogeneous experts are **concatenated** and then each expert block is scaled by task-specific attention weights. Output dimension is the sum of all expert output dimensions.

$$\boldsymbol{w}_k = \mathrm{Softmax}(\boldsymbol{W}_k \cdot \boldsymbol{h}_{\text{shared}} + \boldsymbol{b}_k) \in \mathbb{R}^8$$

$$\tilde{\boldsymbol{h}}_{k,i} = w_{k,i} \cdot \boldsymbol{h}_i^{\text{expert}} \quad \text{for } i = 1, \ldots, 8$$

$$\boldsymbol{h}_k^{\text{cgc}} = \left[\tilde{\boldsymbol{h}}_{k,1} \parallel \tilde{\boldsymbol{h}}_{k,2} \parallel \ldots \parallel \tilde{\boldsymbol{h}}_{k,8}\right] \in \mathbb{R}^{576}$$

> $\boldsymbol{W}_k \in \mathbb{R}^{8\times 576}$: gate weight for task k
> $w_{k,i}$: attention weight task k assigns to Expert i
> Result: same 576D but with different expert contribution weights per task

In this PLE-Cluster-adaTT implementation, **CGCAttention** (block scaling) is used for 8 heterogeneous Shared Experts. Analogous to Transformer Attention, the relationship between **Query** (gate weight), **Key** (shared representation), and **Value** (Expert output) enables "selectively combining information in proportion to relevance."

Initial Bias Setting (domain_experts)

The per-task `domain_experts` config assigns `bias_high=1.0` to preferred experts and `bias_low=-1.0` to non-preferred ones, guiding expert selection at the start of training.

Entropy Regularization

Entropy of the gate distribution is regularized to prevent Expert Collapse:

$$\mathcal{L}_{\text{entropy}} = \lambda_{\text{ent}} \cdot \left(-\frac{1}{|\mathcal{T}|}\right) \sum_{k\in\mathcal{T}} H(\boldsymbol{w}_k), \quad H(\boldsymbol{w}_k) = -\sum_{i=1}^{8} w_{k,i} \cdot \log\big(w_{k,i}\big)$$

$\lambda_{\text{ent}} = 0.01$: minimizing negative entropy increases entropy, encouraging distributed expert utilization

Dimension Normalization (v3.3)

Contribution imbalance due to heterogeneous expert dimension asymmetry (128D vs 64D) is corrected:

$$\text{scale}_i = \sqrt{\frac{\text{mean_dim}}{\text{dim}_i}}, \quad \text{mean_dim} = \frac{128 + 64 \times 7}{8} = 72.0$$

unified_hgcn (128D): scale ≈ 0.750 (attenuation), others (64D): scale ≈ 1.061 (amplification)

2. Heterogeneous Expert Basket

2.1 Pool/Basket Pattern

Instead of PLE's identically structured experts, this system combines **8 heterogeneous domain experts** into a Shared Expert Pool. Each expert interprets the input data from an entirely different mathematical perspective, and the CGC Gate learns the optimal per-task combination.

Expert	Input	Output	Role and Irreplaceability
DeepFM	normalized 644D	64D	Explicitly captures FM $O(nk)$ second-order feature interactions
LightGCN	precomputed 64D	64D	Bipartite graph-based "similar customers" collaborative signal
Unified HGCN	27D (merchant_hierarchy, MCC hierarchy)	128D	Encodes MCC hierarchical structure in hyperbolic space via Poincaré embeddings
Temporal	sequence input	64D	Mamba+LNN+Transformer temporal pattern ensemble
PersLay	Persistence Diagram	64D	Topological structure of spending patterns (loops, clusters)
Causal	161D (demographics, products, txn, derived_temporal, product_hierarchy, gmm)	64D	Directional causal relation extraction via SCM/NOTEARS
Optimal Transport	127D (demographics, products, txn, derived_temporal, gmm)	64D	Sinkhorn Wasserstein distributional geometry
RawScale	raw 90D	64D	Preserves power-law distribution information before normalization

Combined dimension: $7 \times 64 + 1 \times 128 = 576$D.

$$\boldsymbol{h}_{\text{shared}} = [\text{unified_hgcn}_{128\text{D}} \parallel \text{perslay}_{64\text{D}} \parallel \text{deepfm}_{64\text{D}} \parallel \text{temporal}_{64\text{D}} \parallel \text{lightgcn}_{64\text{D}} \parallel \text{causal}_{64\text{D}} \parallel \text{OT}_{64\text{D}} \parallel \text{raw_scale}_{64\text{D}}]$$

2.2 Expert Selection Criteria

The 8 experts extract **fundamentally different mathematical structures** from the same customer data:

- **DeepFM/Causal/OT**: receive overlapping but distinct feature subsets (Causal: 161D, OT: 127D) and capture different structures — symmetric interactions (FM), asymmetric causality (DAG), distributional distance (Wasserstein)
- **Temporal**: temporal dynamics of sequential data ($[B, 180, 16]$, $[B, 90, 8]$)
- **PersLay**: topological invariants (Betti numbers, persistence)
- **Unified HGCN**: hierarchical relationships in hyperbolic geometry (merchant_hierarchy, MCC hierarchy; not product_hierarchy)
- **LightGCN**: collaborative filtering signal from the customer-merchant graph; also handles product co-holding collaborative filtering via the bipartite graph
- **RawScale**: raw scale/power-law distribution patterns lost during normalization

2.3 Expert Routing

Each expert is activated in the `shared_experts` section of the config, and different inputs are dispatched per expert name in `_forward_shared_experts()`. When input data is `None`, a **zero tensor fallback** is applied, and CGC gating automatically reduces that expert's weight.

HMM Triple-Mode is handled through a separate input path:

HMM Mode	Input	Time Scale	Target Tasks
Journey	16D	daily	has_nba, cross_sell_count, will_acquire_*
Lifecycle	16D	monthly	churn_signal, product_stability, segment_prediction
Behavior	16D	monthly	nba_primary, next_mcc, mcc_diversity_trend, top_mcc_shift, …

Each mode consists of 10D base state probabilities + 6D ODE dynamics, projected from 16D to 32D and concatenated with the CGC output:

$$\boldsymbol{h}^m_{\text{hmm}} = \text{SiLU}(\text{LayerNorm}(\text{Linear}_{16\to 32}(\boldsymbol{x}^m_{\text{hmm}})))$$

3. adaTT — Adaptive Task-aware Transfer

3.1 Motivation: Negative Transfer

When 13 tasks share Shared Expert parameters, gradient conflicts cause Negative Transfer. Three limitations of fixed-tower MTL:

1. **Unidirectional sharing**: no mechanism to detect or regulate inter-task interference
2. **Unmeasured interaction**: impossible to quantify which task pairs help or harm each other
3. **Temporal change not reflected**: training-stage-wise changes in task relationships cannot be tracked

adaTT resolves these with **selective transfer**, **gradient cosine similarity**, and a **3-Phase schedule**, respectively.

3.2 Gradient Cosine Similarity

For two tasks i, j with gradients with respect to Shared Expert parameters θ, denoted $\boldsymbol{g}_i = \nabla_\theta \mathcal{L}_i$, $\boldsymbol{g}_j = \nabla_\theta \mathcal{L}_j$:

$$\cos(\theta_{i,j}) = \frac{\boldsymbol{g}_i \cdot \boldsymbol{g}_j}{\|\boldsymbol{g}_i\| \cdot \|\boldsymbol{g}_j\|}$$

$\boldsymbol{g}_i \in \mathbb{R}^d$: flattened gradient vector of task i
Result range: $[-1, 1]$ — positive = positive transfer, negative = negative transfer

Reasons for using cosine similarity:
1. **Scale invariance**: compares directions purely without being affected by per-task loss magnitude differences
2. **Interpretability**: normalized to $[-1, 1]$ range for intuitive interpretation
3. **Efficient computation**: L2-normalize gradient matrix $\boldsymbol{G} \in \mathbb{R}^{n \times d}$ then compute all n^2 similarities via a single GEMM as $\hat{\boldsymbol{G}}\hat{\boldsymbol{G}}^\top$

```
norms = grad_matrix.norm(dim=1, keepdim=True).clamp(min=1e-8)
normalized = grad_matrix / norms
affinity = torch.mm(normalized, normalized.t())  # [n_tasks, n_tasks]
```

3.3 EMA Stabilization

Gradients from a single batch are noisy, so EMA smoothing is applied:

$$\boldsymbol{A}_t = \alpha \cdot \boldsymbol{A}_{t-1} + (1 - \alpha) \cdot \cos(\theta_t)$$

$\alpha = 0.9$: effective window $\approx \frac{1}{1-\alpha} = 10$ observations
Signal processing perspective: IIR first-order low-pass filter $H(z) = \frac{1-\alpha}{1-\alpha z^{-1}}$

After EMA update, `.clamp(-1.0, 1.0)` must be applied — accumulated floating-point errors can push cosine similarity outside $[-1, 1]$, potentially causing NaN in downstream operations.

3.4 Transfer-Enhanced Loss

adaTT-enhanced loss for each task i:

$$\mathcal{L}_i^{\text{adaTT}} = \mathcal{L}_i + \lambda \cdot \sum_{j \neq i} w_{i \to j} \cdot \mathcal{L}_j$$

$\mathcal{L}_i$: original loss for task i
$\lambda = 0.1$: proportion of other tasks' opinions incorporated (10%)
$w_{i \to j}$: transfer weight from task j to task i (softmax-normalized)

The system also supports sigmoid gating (NeurIPS 2024, Nguyen et al.). Sigmoid gates allow independent expert contribution, eliminating harmful inter-expert competition in heterogeneous expert environments. Configuration: `gate_type: "sigmoid"` (pipeline.yaml or HP).

Effect on gradients:

$$\nabla_\theta \mathcal{L}_i^{\text{adaTT}} = \nabla_\theta \mathcal{L}_i + \lambda \sum_{j \neq i} w_{i \to j} \nabla_\theta \mathcal{L}_j$$

The second term is a **correction vector** that steers shared parameters toward directions favorable to multiple tasks.

3.5 Transfer Weight Computation (4-stage pipeline)

$$\begin{aligned} \boldsymbol{R} &= (\boldsymbol{W} + \boldsymbol{A}) \cdot (1 - r) + \boldsymbol{P} \cdot r \\ \boldsymbol{R}_{i,j} &\leftarrow 0 \quad \text{if } \boldsymbol{A}_{i,j} < \tau_{\text{neg}} \\ \boldsymbol{R}_{i,i} &= 0 \\ w_{i \to j} &= \text{softmax}\left(\frac{\boldsymbol{R}_{i,j}}{T}\right) \end{aligned}$$

$\boldsymbol{W}$: learnable transfer weights (`nn.Parameter` , initialized to 0)
$\boldsymbol{A}$: EMA affinity matrix
$\boldsymbol{P}$: Group Prior matrix (domain knowledge)
r: Prior blend ratio (annealed from $0.5 \to 0.1$)
$\tau_{\text{neg}} = -0.1$: threshold for blocking negative transfer
$T = 1.0$: Softmax temperature

Meaning of each stage:
1. Sum learnable weights $\boldsymbol{W}$ with observed affinity $\boldsymbol{A}$, blend with domain prior $\boldsymbol{P}$

2. Zero out paths where affinity falls below the threshold (negative transfer blocking)
3. Exclude self-transfer (diagonal set to 0)
4. Generate probability distribution via Softmax — scale is constant regardless of task count since values sum to 1

`max_transfer_ratio = 0.5`: Transfer loss cannot exceed 50% of the original loss.

3.6 3-Phase Schedule

Phase	Duration	Behavior	Purpose
1: Warmup	epoch 0 `warmup_epochs`	affinity computation only, no transfer loss applied	Accumulate stable affinity data
2: Dynamic	`warmup_epochs` `freeze_epoch`	transfer active, Prior blend annealing	Learn and apply task relationships
3: Frozen	`freeze_epoch` end	transfer weights frozen (`detach`)	Stabilize fine-tuning, remove gradient overhead

Validation: freeze_epoch > warmup_epochs

If Phase 2 is completely skipped, the learned affinity is never reflected in transfer, rendering adaTT usage meaningless. Validated at initialization with an early `ValueError`.

adaTT Epoch Budget

With 10-epoch runs (warmup=3, freeze=8), adaTT has a limited dynamic window of only 5 epochs (Phase 2: epochs 3–8). This is often insufficient for affinity to converge. **20-epoch runs are recommended for meaningful adaTT evaluation** — they provide a 12-epoch dynamic window (warmup=3, freeze=15) which allows affinity matrices to stabilize before freezing.

3.7 Group Prior

The Prior blend ratio decreases linearly as training progresses:

$$r(e) = r_{\text{start}} - (r_{\text{start}} - r_{\text{end}}) \cdot \min\left(\frac{e - e_{\text{warmup}}}{e_{\text{freeze}} - e_{\text{warmup}}}, 1.0\right)$$

$r_{\text{start}} = 0.5 \rightarrow r_{\text{end}} = 0.1$: rely on domain knowledge early in training, trust observed affinity later

Bayesian interpretation: transition from Prior ($\boldsymbol{P}$) to Posterior (observation-based $\boldsymbol{A}$). Rely on prior when data is scarce early on, then follow likelihood as data accumulates.

3.8 Negative Transfer Detection and Blocking

$$\boldsymbol{R}_{i,j} = \begin{cases} \boldsymbol{R}_{i,j} & \text{if } \boldsymbol{A}_{i,j} > \tau_{\text{neg}} \\ 0 & \text{otherwise} \end{cases}$$

Threshold $\tau_{\text{neg}} = -0.1$ (not zero, because): weak negative correlation may be noise and is therefore tolerated; only clearly opposing gradient directions are blocked. Setting to 0 would over-prune paths and weaken adaTT effectiveness.

Diagnostic API: `detect_negative_transfer()` returns negative transfer pairs in the form `{"churn_signal": ["will_acquire_lending", "nba_primary"]}`.

3.9 Analogy to Attention Mechanism

adaTT can be interpreted as **Attention in task space**:

Role	Transformer Self-Attention	adaTT Task Transfer
Query	current token's query	current task's gradient direction
Key	other tokens' response capacity	other tasks' gradient directions
Similarity	$\boldsymbol{Q}\frac{\boldsymbol{K}^\top}{\sqrt{d_k}}$	gradient cosine similarity
Normalization	softmax	softmax (temperature T)
Value	other tokens' information	other tasks' loss values
Output	weighted context	transfer loss

Compared to Hypernetworks (Ha et al., 2017), adaTT uses **observed gradients** instead of learned task embeddings as conditioning signals, enabling immediate adaptation to changes in task relationships.

4. Task Groups

4.1 Four Financial DNA Groups

Based on domain knowledge, 13 tasks are classified into 4 groups:

Group	Members	Intra	Inter	Business Meaning
Engagement	cross_sell_count, will_acquire_deposits, will_acquire_investments, will_acquire_accounts, will_acquire_lending, will_acquire_payments	0.8	0.3	Customer engagement/ conversion
Lifecycle	churn_signal, product_stability, segment_prediction	0.7	0.3	Customer lifecycle
Value	nba_primary	0.6	0.3	Customer value/behavioral patterns
Consumption	next_mcc, mcc_diversity_trend, top_mcc_shift	0.7	0.3	Spending pattern analysis

Intra-group strength: transfer strength between tasks within the same group. The Engagement group (0.8) is highest — product acquisition tasks share similar structure and are expected to exhibit strong positive transfer.

Inter-group strength: transfer strength across groups. Conservative at 0.3 — cross-group transfer is only activated after verification through gradient observation.

4.2 Intra/Inter Strength Design

The Group Prior matrix $\boldsymbol{P}$ is automatically generated from the group structure:

- Within the same group: intra_strength (0.6 0.8)
- Across different groups: inter_group_strength (0.3)
- Diagonal: 0 (self-transfer excluded)
- Row-normalized so values sum to 1

4.3 Three Logit Transfer Approaches

Three transfer methods for **explicit information passing** between tasks:

Source	Target	Type	Strength	Business Meaning
has_nba	nba_primary	Sequential	0.5	NBA presence → primary product decision

churn_signal	product_stability	Inverse	0.5	Inverse of churn signal ≈ product stability
next_mcc	mcc_diversity_trend	Feature	0.5	Next spend category → diversity trend

Transfer mechanism (residual form):

$$\boldsymbol{h}^t_{\text{tower}} = \boldsymbol{h}^t_{\text{expert}} + \alpha \cdot \text{SiLU}(\text{LayerNorm}(\text{Linear}(\text{pred}^s)))$$

$\alpha = 0.5$: transfer strength. Transfer is naturally ignored if projection weights converge to zero.

The execution order is automatically derived from dependency relationships in the `task_relationships` config using **Kahn's algorithm** (topological sort). A fallback order is applied if a cycle is detected.

4.4 GroupTaskExpertBasket (v3.2)

88% parameter reduction compared to independent per-cluster×task MLPs (∼3.0M → ∼362K). Tasks in the same Task Groups share a GroupEncoder, differentiated by ClusterEmbedding:

$$\boldsymbol{e}_{\text{cluster}} = \text{Embedding}(\text{cluster_id}) \in \mathbb{R}^{32}$$

$$\boldsymbol{x}_{\text{input}} = \left[\text{CGC_output}_{576\text{D}} \parallel \text{HMM_proj}_{32\text{D}} \parallel \boldsymbol{e}_{\text{cluster}_{32\text{D}}}\right] \in \mathbb{R}^{640}$$

$$\boldsymbol{h}_{\text{expert}} = \text{MLP}_{640\to128\to64\to32}\left(\boldsymbol{x}_{\text{input}}\right)$$

Soft Routing

For cluster boundary samples, embeddings are mixed using GMM posterior probabilities:

$$\boldsymbol{e}_{\text{cluster}} = \sum_{c=0}^{19} p_c \cdot \boldsymbol{E}_c \in \mathbb{R}^{32}$$

p_c: GMM posterior probability, implementation: `cluster_probs @ embedding.weight` ($[B, 20] \times [20, 32]$)

Unlike hard assignment, boundary customer predictions are insensitive to fluctuations in cluster assignments.

5. Training Strategy

5.1 2-Phase Training

Phase 1: Shared Expert Pretrain

- **Duration**: `shared_expert_epochs` (default 15)
- **Training target**: entire model — Shared Experts, CGC, Task Experts, Task Towers
- **adaTT**: active — gradient extraction and transfer loss applied

Phase 2: Cluster Finetune

- **Duration**: `cluster_finetune_epochs` (default 8)
- **Training target**: per-cluster Task Expert sub-heads only
- **adaTT**: **inactive** — gradient extraction is meaningless since Shared Expert is frozen
- **CGC**: frozen — learning gating is unnecessary and causes overfitting since inputs (Expert outputs) do not change

Items reset at phase transition:

Reset Item	Reason
Optimizer	Shared Expert frozen → prevent stale momentum
Scheduler	Phase 2-specific warmup (2 epochs, shorter than Phase 1's 5 epochs)
GradScaler	AMP scaler state initialization (loss scale changes)
Early stopping	reset best_val_loss, patience_counter
CGC Attention	Shared Expert frozen → freeze CGC together

adaTT is always restored after Phase 2 ends (guaranteed even on exceptions via `finally` block).

5.2 Uncertainty Weighting

Kendall et al., CVPR 2018 Automatic task weighting based on homoscedastic uncertainty:

$$\mathcal{L}_i^{\text{weighted}} = \frac{1}{2\sigma_i^2} \cdot \mathcal{L}_i + \frac{1}{2} \log \sigma_i^2$$

$\sigma_i^2 = \exp(\log\backslash_\text{var}_i)$: learnable uncertainty for task i

`log_var` clamp: $[-4.0, 4.0]$, precision clamp: $[0.001, 100.0]$

Tasks with high uncertainty (σ_i^2 large) automatically receive lower weight ($\frac{1}{2\sigma_i^2}$), and the $\log \sigma_i^2$ regularization term prevents uncertainty from growing unboundedly. This **automatic balancing** replaces the combinatorial explosion of manually tuning weights for 13 tasks.

Uncertainty Weighting is applied **before** adaTT. The `task_losses` input to adaTT already reflects uncertainty weighting.

5.3 Focal Loss

Lin et al., ICCV 2017 Loss for class-imbalanced binary classification:

$$\mathrm{FL}(p_t) = -\alpha_t \cdot (1 - p_t)^\gamma \cdot \log(p_t)$$

The $(1 - p_t)^\gamma$ term is key — it sharply reduces the loss for easy examples ($p_t \approx 1$) and focuses learning on hard examples ($p_t \approx 0$).

Task	weight	Focal α/γ	Notes
will_acquire_*	1.5	γ=2, α=0.20	extremely low positive rate → weight increased
churn_signal	1.2	γ=2, α=0.60	high FN cost → alpha increased
nba_primary	2.0	CE (multiclass)	business critical
next_mcc	2.0	CE (multiclass)	spending category prediction

5.4 AMP (Automatic Mixed Precision)

Forward pass runs in fp16 with `torch.amp.autocast`. Additional 10 15% speed improvement with TF32 + cuDNN benchmark:

```
torch.backends.cuda.matmul.allow_tf32 = True
torch.backends.cudnn.allow_tf32 = True
torch.backends.cudnn.benchmark = True
```

LR scheduler: Linear Warmup (5 epochs, start_factor=0.1) → CosineAnnealingWarmRestarts (T_0=10, T_{mult}=2). Scheduler is reset in Phase 2 (2-epoch warmup).

6. Implementation Notes

6.1 FP16 Considerations

Focal Loss float32 Casting

In fp16, intermediate results of `focal_weight * bce` may fall into subnormal range, potentially causing NaN. The entire focal loss computation is performed in float32:

```
p_f = pred.squeeze().float().clamp(1e-7, 1 - 1e-7)
```

adaTT Gradient Extraction and autocast

adaTT gradient extraction takes place inside autocast, but the loss computation itself is cast to float32 to ensure numerical stability.

6.2 GradScaler Guard

`retain_graph=True` cannot be removed due to architectural constraints — `_extract_task_gradients` is called after loss computation and before `backward()`, and the same computation graph must be reused in the Trainer's `loss.backward()`.

Memory impact:

Component	Memory	Notes
Forward pass graph	1x	baseline
retain_graph overhead	~1x	additional memory due to graph not being freed
13 task gradients	~0.3x	each gradient is shared_param_size
Total	**~2.3x**	batch 16384 possible on RTX 4070 12GB

Gradient Extraction Frequency Optimization

`adatt_grad_interval=10`: gradient extraction every 10 steps instead of every step. Sufficiently stable thanks to EMA smoothing, with 1/10 reduction in computational overhead.

torch.compiler.disable

`@torch.compiler.disable` decorator applied to `_extract_task_gradients`. `torch.autograd.grad` has incomplete `requires_grad` tracking inside compiled graphs. `torch.compile` itself is currently disabled (15-task MTL + retain_graph + dynamic shapes cause hundreds of kernel compilations, with first epoch taking 30+ minutes), but applied defensively.

6.3 Phase 2 Frozen Layers

Layers frozen in Phase 2:

Layer	Frozen	Reason
Shared Experts	Yes	Preserve shared representations sufficiently learned in Phase 1
CGC Attention	Yes	Expert outputs fixed → learning gating is unnecessary
adaTT	Inactive	Shared Expert frozen → gradient 0, cosine similarity meaningless
GroupEncoder	No	Per-cluster specialization learning (core of Phase 2)
Task Towers	No	Fine-tune final prediction layers
ClusterEmbedding	No	Refine cluster representations

CGC-adaTT Co-freezing

CGC Attention is also frozen together at adaTT `freeze_epoch`. If CGC continues learning, expert weights change and the affinity relationships measured by adaTT may be invalidated.

```
if freeze_epoch is not None and epoch >= freeze_epoch:
    for param in self.task_expert_attention.parameters():
        param.requires_grad = False
    self._cgc_frozen.fill_(True)
```

`_cgc_frozen` is registered as a `register_buffer` so its state is preserved when saving/restoring checkpoints.

6.4 Full Loss Computation Pipeline Order

1. **Determine per-task loss type** (focal, huber, MSE, NLL, contrastive)
2. **Apply Focal Loss alpha/gamma** (per-task positive class weights)
3. **Apply loss weights** (Uncertainty Weighting or fixed weights)
4. **Add evidential loss** (auxiliary uncertainty estimation loss)
5. **adaTT transfer loss** (add gradient-based transfer loss)
6. **CGC entropy regularization** (prevent Expert Collapse)

6.5 Debugging Guide

Symptom	Cause	Resolution
NaN loss	fp16 focal loss underflow	verify float32 casting
Loss divergence at training start	transfer loss dominating original	check `max_transfer_ratio` (0.5)
Phase 2 RuntimeError	missing adaTT deactivation	verify `model.adatt = None`
`ValueError` at initialization	`freeze_epoch <= warmup_epochs`	validate config

Training hang (no response)	gradient extraction every step	check `adatt_grad_interval` (default 10)
Checkpoint mismatch	`fill_()` not used	verify in-place operation for buffer updates

Characteristics of a healthy affinity matrix:

- Within the same group: positive affinity (> 0.3)
- Across different groups: weakly positive or neutral ($-0.1 \sim 0.3$)
- Diagonal: 1.0
- Not saturated at ± 1 overall (if saturated, adjust EMA decay rate)

References

1. Caruana, R. (1997). Multitask Learning. *Machine Learning*, 28(1).
2. Ma, J. et al. (2018). Modeling Task Relationships in Multi-Task Learning with Multi-Gate Mixture-of-Experts. *KDD*.
3. Tang, H. et al. (2020). Progressive Layered Extraction (PLE): A Novel MTL Model for Personalized Recommendations. *RecSys*.
4. Kendall, A., Gal, Y. & Cipolla, R. (2018). Multi-Task Learning Using Uncertainty to Weigh Losses. *CVPR*.
5. Lin, T.-Y. et al. (2017). Focal Loss for Dense Object Detection. *ICCV*.
6. Yu, T. et al. (2020). Gradient Surgery for Multi-Task Learning (PCGrad). *NeurIPS*.
7. Chen, Z. et al. (2018). GradNorm: Gradient Normalization for Adaptive Loss Balancing. *ICML*.
8. Fifty, C. et al. (2021). Efficiently Identifying Task Groupings for Multi-Task Learning. *NeurIPS*.
9. Navon, A. et al. (2022). Multi-Task Learning as a Bargaining Game (Nash-MTL). *ICML*.
10. Liu, B. et al. (2021). Conflict-Averse Gradient Descent for Multi-Task Learning (CAGrad). *NeurIPS*.
11. Ha, D. et al. (2017). HyperNetworks. *ICLR*.
12. Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS*.
13. Jacobs, R. et al. (1991). Adaptive Mixtures of Local Experts. *Neural Computation*.
14. Fedus, W. et al. (2022). Switch Transformers: Scaling to Trillion Parameter Models. *JMLR*.